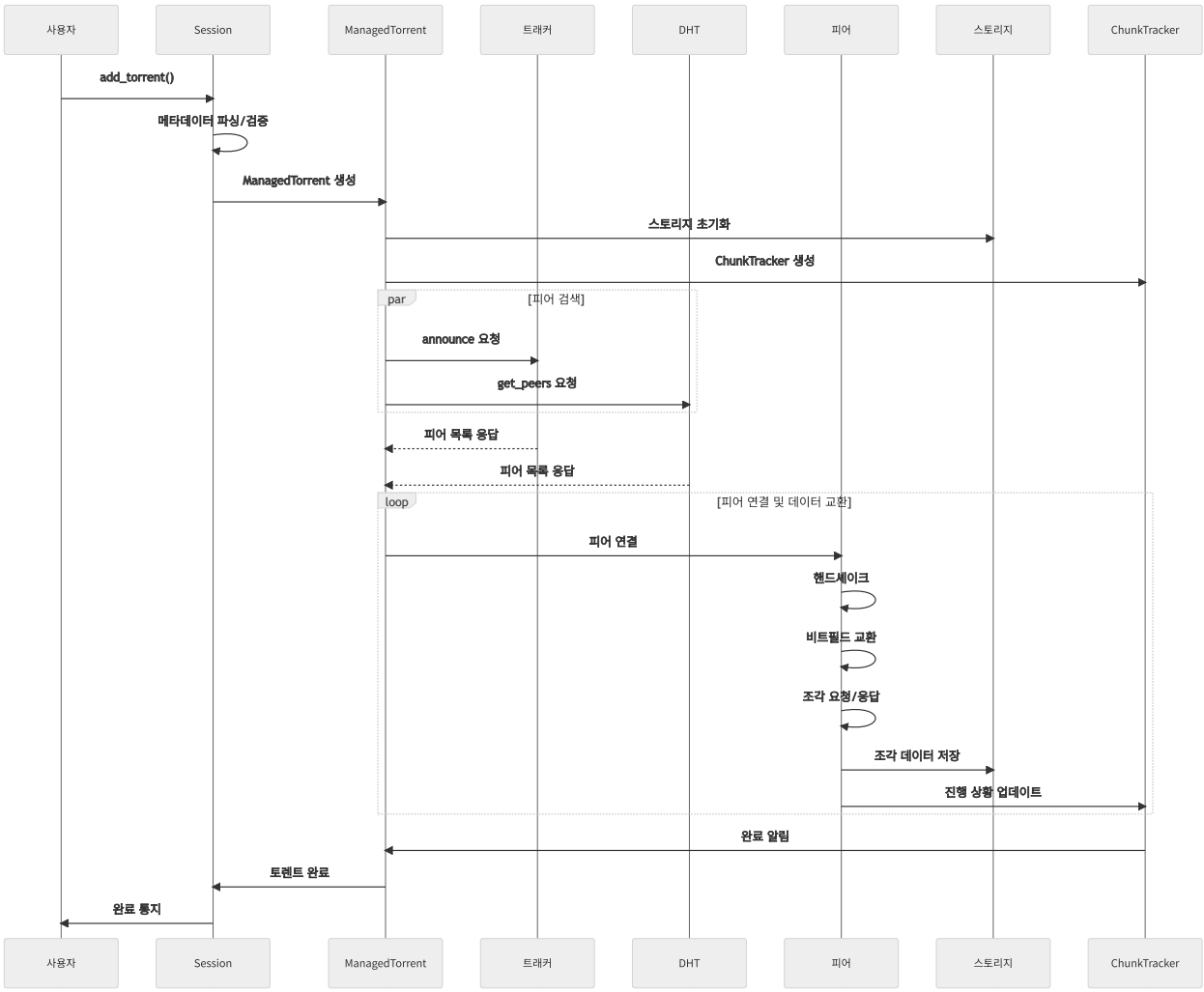
rqbit 토렌트 다운로드 전체 흐름 추적 분석

개요

이 문서는 rqbit BitTorrent 클라이언트에서 토렌트 추가부터 완료까지의 전체 과정을 단계별로 추적하고, 각 단계에서 관련된 모듈들의 상호작용을 분석합니다. 또한 오류 발생 시의 복구 과정과 상태 전이를 상세히 파악합니다.

전체 흐름 다이어그램


사용자
Session
ManagedTorrent
트래커
DHT
피어
스토리지
ChunkTracker
add_torrent()
메타데이터 파싱/검증
ManagedTorrent 생성
스토리지 초기화
ChunkTracker 생성
par
[피어 검색]
announce 요청
get_peers 요청
피어 목록 응답
피어 목록 응답
loop
[피어 연결 및 데이터 교환]
피어 연결
핸드셰이크
비트필드 교환
조각 요청/응답
조각 데이터 저장
진행 상황 업데이트
완료 알림
토렌트 완료
완료 통지
사용자
Session
ManagedTorrent
트래커
DHT
피어
스토리지
ChunkTracker

1단계: 토렌트 추가 및 초기화

1.1 add_torrent 메서드 진입

```
// Session::add_torrent 호출
pub fn add_torrent<'a>(
self: &'a Arc<Self>,
add: AddTorrent<'a>,
opts: Option<AddTorrentOptions>,
) -> BoxFuture<'a, anyhow::Result<AddTorrentResponse>>
```

주요 처리 과정:

1. **입력 검증**: AddTorrent 타입에 따른 분기 처리
   - URL (HTTP/HTTPS): 토렌트 파일 다운로드
   - 마그넷 링크: 메타데이터 해결 필요
   - 바이트 데이터: 직접 파싱
2. **메타데이터 처리**:

```
let torrent = match add {
AddTorrent::Url(url) if url.starts_with("magnet:") => {
// 마그넷 링크 파싱
let magnet = Magnet::parse(&url)?;
let info_hash = magnet.as_id20()?;
// 메타데이터는 나중에 피어로부터 획득
},
AddTorrent::TorrentFileBytes(bytes) => {
// Bencode 디코딩 및 검증
torrent_from_bytes(bytes)?
}
};
```

1.2 토렌트 메타데이터 검증

TorrentMetadata 생성 과정:

```
metadata: Some(TorrentMetadata::new(
torrent.meta.info.data.validate()?, // 토렌트 정보 검증
torrent.torrent_bytes, // 원본 바이트 보존
torrent.meta.info.raw_bytes.0, // info 딕셔너리 원본
)?)
```

검증 항목:

- Bencode 구조 유효성
- 필수 필드 존재 여부 (name, piece length, pieces)
- 해시 길이 검증 (SHA-1: 20바이트 × 조각 수)
- 파일 정보 일관성

1.3 ManagedTorrent 생성

```
let handle = Arc::new(ManagedTorrent {
locked: RwLock::new(ManagedTorrentLocked {
paused: opts.paused,
state: ManagedTorrentState::Initializing(initializing),
only_files,
}),
state_change_notify: Notify::new(),
shared: minfo,
metadata: ArcSwapOption::new(Some(metadata.clone())),
});
```

초기 상태: `ManagedTorrentState::Initializing`

2단계: 초기화 프로세스 (Initializing 상태)

2.1 파일 시스템 준비

TorrentStateInitializing::check 메서드:

```
pub async fn check(
self: Arc<Self>,
bitv_factory: &dyn BitVFactory,
) -> anyhow::Result<TorrentStatePaused>
```

처리 단계:

1. **FastResume 데이터 로드**:

```
let have_pieces = bitv_factory
.load(id)
.await
.context("error loading have_pieces")?;
```

2. **FastResume 검증**:

- 비트필드 길이 검증
- 무작위 조각 샘플링 (10개)
- 각 파일당 최소 1개 조각 검증

3. **전체 무결성 검사** (FastResume 실패 시):

```
let have_pieces = FileOps::new(&self.metadata.info, &self.files, &self.metadata.
.initial_check(&self.checked_bytes);
```

2.2 ChunkTracker 생성

```
let chunk_tracker = ChunkTracker::new(
have_pieces.into_dyn(), // 보유 조각 비트필드
selected_pieces, // 선택된 조각들
*self.metadata.lengths(), // 길이 정보
&self.metadata.file_infos, // 파일 정보
);
```

ChunkTracker 초기화:

- `queue_pieces` : 다운로드 대기 조각들 (`selected & !have`)
- `chunk_status` : 청크별 다운로드 상태
- `per_file_bytes` : 파일별 다운로드된 바이트 수

2.3 상태 전이: Initializing → Paused

```
Ok(TorrentStatePaused {
shared: self.shared.clone(),
metadata: self.metadata.clone(),
files: self.files,
chunk_tracker,
streams: Arc::new(TorrentStreams::new()),
})
```

3단계: 피어 검색 및 연결 설정

3.1 피어 스트림 생성

make_peer_rx 함수에서 다중 소스 통합:

```
fn make_peer_rx(
info_hash: Id20,
trackers: Vec<url::Url>,
announce: bool,
initial_peers: Vec<SocketAddr>,
is_private: bool,
) -> Option<PeerStream>
```

피어 소스들:

1. **DHT 피어 검색** (프라이빗 토렌트 제외):

```
let dht_rx = self.dht.as_ref().map(|dht| {
dht.get_peers(info_hash, if announce { self.announce_port } else { None })
});
```

2. **트래커 통신**:

```
let tracker_rx = TrackerComms::start(
info_hash,
self.peer_id,
trackers.into_iter().collect(),
// ... 기타 매개변수
```

```
);
```

3. **로컬 서비스 검색 (LSD)**:

```
let lsd_rx = self.lsd.as_ref().and_then(|lsd| {
lsd.get_peers(info_hash)
});
```

3.2 스트림 병합

```
merge_two_optional_streams(
merge_two_optional_streams(
merge_two_optional_streams(dht_rx, tracker_rx),
initial_peers_rx,
),
lsd_rx,
)
```

특징:

- 모든 소스에서 동시에 피어 수집
- 빠른 응답 우선 처리
- 중복 피어 자동 제거

4단계: 라이브 상태 전환 및 피어 관리

4.1 상태 전이: Paused → Live

```
pub(crate) fn unpause(&self) -> anyhow::Result<()> {
let mut g = self.locked.write();
match g.state.take() {
ManagedTorrentState::Paused(paused) => {
let live = paused.into_live()?;
g.state = ManagedTorrentState::Live(live);
g.paused = false;
}
// ... 기타 상태 처리
}
self.state_change_notify.notify_waiters();
Ok(())
}
```

4.2 TorrentStateLive 아키텍처

핵심 구성 요소:

```
pub struct TorrentStateLive {
shared: Arc<ManagedTorrentShared>,
state: Arc<TorrentStateLocked>,
peers: Arc<PeerStates>,
```

```
spawner: BlockingSpawner,
// ... 기타 필드들
}
```

주요 태스크들:

1. **피어 추가 태스크**: 새로운 피어 연결 관리
2. **피어 관리 태스크**: 개별 피어와의 통신
3. **청크 요청 태스크**: 피어별 조각 요청 관리

4.3 피어 생명주기 관리

```rust
#[derive(Debug, Default)]
pub(crate) enum PeerState {
#[default]
Queued, // 연결 대기 중
Connecting(PeerTx), // 연결 시도 중
Live(LivePeerState), // 활성 연결
Dead, // 오류로 인한 비활성 상태
NotNeeded, // 더 이상 필요하지 않음
}
```

상태 전이 흐름:

```
Queued → Connecting → Live → Dead/NotNeeded
↑ ↓ ↓ ↓
```

└──────────┴──────────┴──────────┘

5단계: 데이터 교환 및 다운로드

5.1 핸드셰이크 과정

아웃바운드 연결:

```
// 1. 피어에 연결
let (ckind, mut read, mut write) = self.connector.connect(self.addr).await?;

// 2. 핸드셰이크 전송
let handshake = Handshake::new(self.info_hash, self.peer_id);
let hsz = handshake.serialize_unchecked_len(&mut *write_buf);
write.write_all(&write_buf[..hsz]).await?;

// 3. 상대방 핸드셰이크 수신 및 검증
let h = read_buf.read_handshake(&mut read, rwtimeout).await?;
if h.info_hash != self.info_hash {
return Err(Error::WrongInfoHash);
}
```

5.2 비트필드 교환

```
// 비트필드 전송 (보유 조각 정보)
if self.handler.should_send_bitfield() {
let len = self.handler.serialize_bitfield_message_to_buf(&mut *write_buf)?;
write.write_all(&write_buf[..len]).await?;
}


// Unchoke 메시지 전송 (업로드 허용)
let len = Message::Unchoke.serialize(&mut *write_buf, &Default::default())?;
write.write_all(&write_buf[..len]).await?;
```

5.3 조각 요청 메커니즘

task_peer_chunk_requester 태스크:

```
async fn task_peer_chunk_requester(&self) -> crate::Result<()> {
loop {
// 1. 토렌트 완료 확인
if self.state.is_finished_and_no_active_streams() {
return Ok(());
}


// 2. 관심 표시 및 unchoke 대기
update_interest(self, true)?;
self.wait_for_unchoke().await;
```

```
// 3. 조각 선택 전략
let next = self.try_steal_old_slow_piece(10.)
.map_or_else(|| self.reserve_next_needed_piece(), |v| Ok(Some(v)))?
.or_else(|| self.try_steal_old_slow_piece(3.));

// 4. 청크별 요청 생성
for chunk in self.state.lengths.iter_chunk_infos(next) {
let request = Request {
index: next.get(),
begin: chunk.offset,
length: chunk.size,
};

// 5. 속도 제한 및 흐름 제어
self.state.ratelimits
.prepare_for_download(NonZeroU32::new(request.length).unwrap())
.await?;
self.requests_sem.acquire().await?;

// 6. 요청 전송
self.tx.send(WriterRequest::Message(Message::Request(request)))?;
}
```

```
}

}
```

5.4 조각 수신 및 저장

on_received_piece 처리:

```
fn on_received_piece(&self, piece: Piece<ByteBuf<'_>>) -> anyhow::Result<()> {

// 1. 조각 검증

let piece_index = self.state.lengths.validate_piece_index(piece.index)?;

let chunk_info = self.state.lengths.chunk_info_from_received_data(

piece_index, piece.begin, piece.len().try_into()?

)?;



// 2. 인플라이트 요청에서 제거

self.state.peers.with_live_mut(self.addr, "inflight_requests.remove", |h| {

if !h.inflight_requests.remove(&chunk_info) {

anyhow::bail!("peer sent us a piece we did not ask");

}

Ok(())

})?;



// 3. 디스크에 쓰기

write_to_disk(&self.state, self.addr, &self.counters, &piece, &chunk_info)?;
```

```
// 4. 통계 업데이트
self.counters.fetched_bytes.fetch_add(piece.len() as u64, Ordering::Relaxed);
Ok(())
}
```

5.5 디스크 쓰기 및 검증

write_to_disk 함수:

```
fn write_to_disk(
state: &TorrentStateLocked,
addr: SocketAddr,
counters: &AtomicPeerCounters,
piece: &Piece<ByteBuf<'_>>,
chunk_info: &ChunkInfo,
) -> anyhow::Result<()> {
// 1. ChunkTracker에 청크 완료 마킹
let chunk_result = state.lock_write("mark_chunk_downloaded")
.get_chunks_mut()?
.mark_chunk_downloaded(piece);

match chunk_result {
Some(ChunkMarkingResult::Completed) => {
// 2. 조각 완료 시 해시 검증
```

```
let piece_index = chunk_info.piece_index;

let expected_hash = state.torrent().info.pieces.get(piece_index.get() as usize);

// 3. 해시 계산 및 비교

let actual_hash = calculate_piece_hash(state, piece_index)?;

if actual_hash == expected_hash {

// 4. 검증 성공 시 조각 완료 마킹

state.lock_write("mark_piece_downloaded")

.get_chunks_mut()?

.mark_piece_downloaded(piece_index);

} else {

// 5. 검증 실패 시 조각 손상 마킹

state.lock_write("mark_piece_broken")

.get_chunks_mut()?

.mark_piece_broken_if_not_have(piece_index);

}

},

Some(ChunkMarkingResult::NotCompleted) => {

// 조각 아직 미완료

},

Some(ChunkMarkingResult::PreviouslyCompleted) => {

// 이미 완료된 조각 (중복 수신)

},

None => {
```

```
// 잘못된 조각 정보
anyhow::bail!("invalid chunk info");
}
}
Ok(())
}
```

6단계: 진행 상황 추적 및 통계

6.1 ChunkTracker 업데이트

조각 완료 시 통계 업데이트:

```
pub fn mark_piece_downloaded(&mut self, idx: ValidPieceIndex) {
let id = idx.get() as usize;
if !self.have.as_slice()[id] {
self.have.as_slice_mut().set(id, true);
let len = self.lengths.piece_length(idx) as u64;
self.hns.have_bytes += len;
if self.selected[id] {
self.hns.needed_bytes -= len;
}
}
```

```
}
```

6.2 파일별 진행률 업데이트

```
pub fn update_file_have_on_piece_completed(
&mut self,
piece_id: ValidPieceIndex,
file_id: usize,
file_info: &FileInfo,
) -> u64 {
let diff_have = self.lengths.size_of_piece_in_file(
piece_id.get(),
file_info.offset_in_torrent,
file_info.len,
);
self.per_file_bytes[file_id] += diff_have;
file_info.len.saturating_sub(self.per_file_bytes[file_id])
}
```

6.3 속도 추정 및 ETA 계산

SpeedEstimator 업데이트:

```
// 1초마다 속도 업데이트
```

```
let fetched = s.stats.counters.fetched_bytes.load(Ordering::Relaxed);

s.stats.down_speed_estimator.add_snapshot(fetched, None, now);



// ETA 계산

let remaining_bytes = chunk_tracker.hns().needed_bytes;

let download_speed = speed_estimator.current_speed();

let eta = if download_speed > 0 {

Some(Duration::from_secs(remaining_bytes / download_speed))

} else {

None

};
```

7단계: 완료 처리

7.1 토렌트 완료 감지

```
pub const fn finished(&self) -> bool {

self.needed_bytes == 0

}



// 완료 확인 루프

if self.state.is_finished_and_no_active_streams() {

update_interest(self, false)?;
```

```
// 피어와의 상호 관심 확인 후 연결 종료
return Ok(());
}
```

7.2 완료 알림

```
// ManagedTorrent에서 완료 알림
self.completed_notify.notify_waiters();

// 세션 레벨 완료 처리
if let Some(persistence) = self.persistence.as_ref() {
persistence.store_completed(id).await?;
}
```

오류 처리 및 복구 메커니즘

1. 네트워크 오류 복구

피어 연결 실패:

```
match self.connector.connect(self.addr).await {
Ok((ckind, read, write)) => {
// 연결 성공
},
Err(e) => {
```

```
// 연결 실패 시 상태를 Dead로 변경
self.state.peers.with_live_mut(self.addr, "connection_failed", |peer| {
peer.state = PeerState::Dead;
});
// 지수 백오프 후 재시도
tokio::time::sleep(backoff_duration).await;
}
}
```

트래커 통신 실패:

```
// 트래커 응답 오류 시 다른 트래커로 전환
for tracker_url in trackers.iter() {
match announce_to_tracker(tracker_url).await {
Ok(response) => break,
Err(e) => {
warn!("Tracker {} failed: {}", tracker_url, e);
continue;
}
}
}
```

2. 데이터 무결성 오류 복구

해시 검증 실패:

```
if actual_hash != expected_hash {
warn!("Piece {} hash mismatch, marking as broken", piece_index);
// 1. 조각을 손상으로 마킹
state.lock_write("mark_piece_broken")
.get_chunks_mut()?
.mark_piece_broken_if_not_have(piece_index);
// 2. 해당 피어를 의심스러운 피어로 마킹
state.peers.with_live_mut(addr, "mark_suspicious", |peer| {
peer.suspicious_pieces += 1;
if peer.suspicious_pieces > 3 {
peer.state = PeerState::Dead;
}
});
// 3. 다른 피어에서 재다운로드
// (ChunkTracker가 자동으로 대기열에 추가)
}
```

3. 리소스 부족 오류 복구

디스크 공간 부족:

```
match storage.write_chunk(chunk_info, data).await {
Ok(()) => {
```

```
// 쓰기 성공
},
Err(StorageError::DiskFull) => {
// 토렌트를 오류 상태로 전환
self.stop_with_error(anyhow!("Disk full"));
return Err(Error::DiskFull);
},
Err(e) => {
// 기타 스토리지 오류
warn!("Storage error: {}", e);
// 재시도 또는 오류 상태 전환
}
}
```

메모리 부족:

```
// 동시 초기화 세마포어를 통한 메모리 사용량 제어
let _permit = self.concurrent_initialize_semaphore.acquire().await?;

// 요청 세마포어를 통한 인플라이트 요청 수 제한
let _permit = self.requests_sem.acquire().await?;
```

4. 상태 전이 오류 복구

잘못된 상태 전이:

```
fn stop_with_error(&self, error: anyhow::Error) {
let mut g = self.locked.write();
match g.state.take() {
ManagedTorrentState::Live(live) => {
// Live 상태에서 오류 발생 시 정리
if let Err(err) = live.pause() {
warn!("Error pausing live torrent during error handling: {}", err);
}
},
ManagedTorrentState::Initializing(_) => {
// 초기화 중 오류 발생 시 정리
},
_ => {}
};
g.state = ManagedTorrentState::Error(error);
self.state_change_notify.notify_waiters();
}
```

성능 최적화 포인트

1. 메모리 효율성

- **제로카피 데이터 전송**: `ByteBuf` 와 이중 버퍼 사용
- **비트벡터 압축**: 조각 상태를 비트 단위로 저장
- **Arc 공유**: 메타데이터와 설정의 참조 카운팅

2. 네트워크 효율성

- **파이프라이닝**: 여러 요청 동시 처리 (최대 128개)
- **적응형 청킹**: 네트워크 상황에 맞는 요청 크기
- **연결 재사용**: 피어 연결 풀링

3. 디스크 I/O 효율성

- **비동기 쓰기**: 논블로킹 디스크 I/O
- **배치 처리**: 여러 청크를 한 번에 처리
- **메모리 매핑**: 대용량 파일의 효율적 접근

4. CPU 효율성

- **해시 계산 최적화**: 하드웨어 가속 활용
- **비트 연산**: 효율적인 상태 확인
- **캐시 친화적**: 지역성을 고려한 데이터 구조

결론

rqbit의 토렌트 다운로드 전체 흐름은 다음과 같은 특징을 가집니다:

1. **체계적인 상태 관리**: 명확한 상태 전이와 생명주기 관리
2. **강력한 오류 복구**: 다양한 오류 상황에 대한 복구 메커니즘
3. **효율적인 리소스 활용**: 메모리, 네트워크, 디스크 I/O 최적화

4. **확장 가능한 아키텍처**: 모듈식 설계와 플러그인 지원

5. **실시간 모니터링**: 상세한 진행 상황 추적과 통계

이러한 설계를 통해 rqbit은 안정적이고 효율적인 BitTorrent 다운로드를 제공하며, 다양한 네트워크 환경과 오류 상황에서도 견고하게 동작합니다.